

字符串是存储在内存的连续字节中的一系列字符。C++ 处理字符串的方式有两种，一种来自 C 语言，常被称为 C-风格字符串，另一种是基于 string 类库的字符串处理方式。C 风格字符串的处理可以参考 <https://www.cnblogs.com/tongye/p/10688941.html>，本文着重介绍 string 类库的使用。

## 一、string 类简介

C++ 中提供了专门的头文件 string（注意不是 string.h，这个是 C 风格字符串相关函数的头文件），来支持 string 类型。string 类定义隐藏了字符串的数组性质，让我们可以像处理普通变量那样处理字符串。**string** 对象和字符数组之间的主要区别是：可以将 **string** 对象声明为简单变量，而不是数组。

### 1.1 string 类几种常见的构造函数：

1) string(const char \*s)：将 string 对象初始化为 s 指向的字符串

```
string str("Hello!");
```

2) string(size\_type n, char c)：创建一个包含 n 个元素的 string 对象，其中每个元素都被初始化为字符 c

```
string str(10, 'a');
```

3) string(const string &str)：将一个 string 对象初始化为 string 对象 str（复制构造函数）

```
string str1("hello!");  
string str2(str1);
```

4) string()：创建一个默认的 string 对象，长度为 0（默认构造函数）

```
string str;           // 创建一个空的 string 对象
```

**string** 类的设计允许程序自动处理 **string** 的大小，因此，上述代码创建了一个长度为 0 的 string 对象，但是向 str 中写入数据时，程序会自动调整 str 的长度。因此，与使用数组相比，使用 string 对象更方便，也更安全。

### 1.2 用 C 语言风格初始化 string 对象：

C++ 允许使用 C 语言风格来初始化 string 对象：

```
string str = "hello!";
```

## 二、获取 **string** 对象的长度

在 C 语言中，使用 `strlen` 函数获取字符串的长度。在 C++ 中，可以使用 **`string.size()`** 函数或 **`string.length()`** 函数来获得 `string` 对象的长度。在 C++ 标准库中，两者的源代码如下：



```
size_type  __CLR_OR_THIS_CALL  length()  const
{ //  return  length  of  sequence
return    (_Mysize);
}
```

```
size_type  __CLR_OR_THIS_CALL  size()    const
{ //  return  length  of  sequence
return    (_Mysize);
}
```



可见，这两个方法是完全一样的，并没有区别。`length()` 方法是 C 语言习惯保留的，`size()` 方法则是为了兼容 STL 容器而引入的。

```
string str("Hello,World!");
int strLen1 = str.length();
int strLen2 = str.size();
```

## 三、复制 **string** 对象

在 C 语言中，使用 `strcpy`、`strncpy` 函数来实现字符串的复制。在 C++ 中则方便很多，可以直接将一个 `string` 对象赋值给另一个 `string` 对象，即：

```
string str1("Hello,World!");
```

```
string str2;  
str2 = str1;
```

由于 string 类会自动调整对象的大小，因此不需要担心目标数组不够大的问题。

## 四、string 对象的拼接和附加

在 C 语言中，使用 strcat、strncat 函数来进行字符串拼接操作。在 C++ 中也有多种方法来实现字符串拼接和附加操作：

### 4.1 使用 + 操作符拼接两个字符串

```
string str1("hello ");  
string str2("world!");  
string str3 = str1 + str2;
```

### 4.1 使用 += 操作符在字符串后面附加内容

可以使用 += 来在一个 string 对象后面附加一个 string 对象、字符以及 C 风格的字符串：



```
string str1("hello ");  
string str2("world!\n");  
  
str1 += str2;  
str1 += "nice job\n";  
str1 += 'a';
```



### 4.2 使用 string.append() 函数

可以使用 string.append() 函数来在一个 string 对象后面附加一个 string 对象或 C 风格的字符串：

```
string str1 = "hello,world!";
```

```
string str2 = "HELLO,WORLD!";

str1.append(str2);
str1.append("C string");
```

### 4.3 使用 `string.push_back()` 函数

可以使用 `string.push_back()` 函数来在一个 `string` 对象后面附加一个字符：

```
string str("Hello");
str.push_back('a');
```

## 五、`string` 对象的比较

在 C 语言中，使用 `strcmp`、`strncmp` 函数来进行字符串的比较。在 C++ 中，由于将 `string` 对象声明为了简单变量，故而对字符串的比较操作十分简单了，直接使用关系运算符（`==`、`!=`、`<`、`<=`、`>`、`>=`）即可：



```
#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str1("hello");
    string str2("hello");

    if (str1 == str2)
        cout << "str1 = str2" << endl;
    else if (str1 < str2)
```

```

        cout << "str1 < str2" << endl;
    else
        cout << "str1 > str2" << endl;

    return 0;
}

```



当然，也可以使用类似 `strcmp` 的函数来进行 `string` 对象的比较，`string` 类提供的是 **`string.compare()`** 方法，函数原型如下：

```

int compare(const string&str) const;

int compare (size_t pos, size_t len, const string&str) const;           // 参数 pos 为比较字符串中第一个字符的位置，len 为比较字符串的长度

int compare (size_t pos, size_t len, const string&str, size_t subpos, size_t sublen) const;

int compare (const char * s) const;

int compare (size_t pos, size_t len, const char * s) const;

int compare (size_t pos, size_t len, const char * s, size_t n) const;

```



`compare` 方法的返回值如下：

- 1) 返回 0，表示相等；
- 2) 返回结果小于 0，表示比较字符串中第一个不匹配的字符比源字符串小，或者所有字符都匹配但是比较字符串比源字符串短；
- 3) 返回结果大于 0，表示比较字符串中第一个不匹配的字符比源字符串大，或者所有字符都匹配但是比较字符串比源字符串长。

## 六、使用 **string.substr()** 函数来获取子串

可以使用 `string.substr()` 函数来获取子串，`string.substr()` 函数的定义如下：

```
string substr (size_t pos = 0, size_t len = npos) const;
```

其中，`pos` 是子字符串的起始位置（索引，第一个字符的索引为 0），`len` 是子串的长度。这个函数的功能是：复制一个 `string` 对象中从 `pos` 处开始的 `len` 个字符到 `string` 对象 `substr` 中去，并返回 `substr`。

```
string str("Hello,World!");  
string subStr = str.substr(3,5);  
cout << subStr << endl;
```

这段代码的输出结果为："lo,Wo"。

## 七、访问 **string** 字符串的元素

可以像 C 语言中一样，将 `string` 对象当做一个数组，然后使用数组下标的方式来访问字符串中的元素；也可以使用 `string.at(index)` 的方式来访问元素（索引号从 0 开始）：

```
string str("Hello,World!");  
cout << str[1] << endl;           // 使用数组下标的方式访问 string 字符串的元素  
cout << str.at(1) << endl;        // 使用 at 索引访问 string 字符串的元素
```

## 八、**string** 对象的查找操作

### 8.1 使用 **string.find()** 方法查找字符

`find` 方法的函数原型如下：

1) 从字符串的 `pos` 位置开始（若不指定 `pos` 的值，则默认从索引 0 处开始），查找子字符串 `str`。如果找到，则返回该子字符串首次出现时其首字符的索引；否则，

返回 `string::npos` :

```
size_type find (const string& str, size_type pos = 0) const;
```

2) 从字符串的 `pos` 位置开始 (若不指定 `pos` 的值, 则默认从索引 0 处开始), 查找子字符串 `s`。如果找到, 则返回该子字符串首次出现时其首字符的索引; 否则, 返回 `string::npos` :

```
size_type find (const char *s, size_type pos = 0) const;
```

3) 从字符串的 `pos` 位置开始 (若不指定 `pos` 的值, 则默认从索引 0 处开始), 查找 `s` 的前 `n` 个字符组成的子字符串。如果找到, 则返回该子字符串首次出现时其首字符的索引; 否则, 返回 `string::npos` :

```
size_type find (const char *s, size_type pos, size_type n);
```

4) 从字符串的 `pos` 位置开始 (若不指定 `pos` 的值, 则默认从索引 0 处开始), 查找字符 `ch`。如果找到, 则返回该字符首次出现的位置; 否则, 返回 `string::npos` :

```
size_type find (char ch, size_type pos = 0) const;
```

举个查找子字符串的例子 (查找字符的代码与这一样, 只需要将 `find` 函数的参数换成字符即可) :



```
#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str("cat,dog,cat,pig,little cat,hotdog,little pig,angry dog");
    size_t catPos = str.find("cat",0);

    if (catPos == string::npos) {
        printf("没有找到字符串\n");
        return 0;
    }
}
```

```

    }

    while (catPos != string::npos) {
        cout << "在索引 " << catPos << " 处找到字符串" << endl;
        catPos = str.find("cat", catPos + 1);
    }

    return 0;
}

```



程序输出结果如下：

 Microsoft Visual Studio 调试控制台

```

在索引 0 处找到字符串
在索引 8 处找到字符串
在索引 23 处找到字符串

```

## 8.2 string.rfind()

string.rfind() 与 string.find() 方法类似，只是查找顺序不一样，string.rfind() 是从指定位置 pos（默认为字符串末尾）开始向前查找，直到字符串的首部，并返回第一次查找到匹配项时匹配项首字符的索引。换句话说，就是查找子字符串或字符最后一次出现的位置。还是以上面的程序为例，稍作修改：



```

#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str("cat,dog,cat,pig,little cat,hotdog,little pig,angry dog");
    size_t catPos = str.rfind("cat",str.length()-1);
}

```



```

if (catPos == string::npos) {
    printf("没有找到字符串\n");
    return 0;
}

while (catPos != string::npos) {
    cout << "在索引 " << catPos << " 处找到字符串" << endl;
    catPos = str.rfind("cat", catPos - 1);
    if (catPos == 0) {
        cout << "在索引 " << catPos << " 处找到字符串" << endl;
        break;
    }
}

return 0;
}

```



程序输出结果如下：

 Microsoft Visual Studio 调试控制台

```

在索引 23 处找到字符串
在索引 8 处找到字符串
在索引 0 处找到字符串

```

可以看到，rfind 方法是从字符串末开始查找的。

### 8.3 string.find\_first\_of()

string.find\_first\_of() 方法在字符串中从指定位置开始向后（默认为索引 0 处）查找参数中任何一个字符首次出现的位置。举个例子说明：



```
#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str("cat,dog,cat,pig,little cat,hotdog,little pig,angry dog");
    size_t pos = str.find_first_of("zywfgat");

    if (pos == string::npos) {
        printf("没有匹配到\n");
        return 0;
    }
    else
        cout << "在索引 " << pos << " 处匹配到" << endl;

    return 0;
}
```



程序输出结果是：在索引 1 处匹配到。所查找的字符串 zywfgat 中，第一次出现在字符串 str 中的字符是 'a'，该字符在 str 中的索引是 1。

#### 8.4 string.find\_last\_of()

string.find\_last\_of() 方法在字符串中查找参数中任何一个字符最后一次出现的位置（也就是从指定位置开始往前查找，第一个出现的位置）。

#### 8.5 string.find\_first\_not\_of()

string.find\_first\_not\_of() 方法在字符串中查找第一个不包含在参数中的字符。

#### 8.6 string.find\_last\_not\_of()

string.find\_last\_not\_of() 方法在字符串中查找最后一个不包含在参数中的字符（从指定位置开始往前查找，第一个不包含在参数中的字符）。

## 九、string 对象的插入和删除操作

### 9.1 使用 string.insert() 进行插入操作

函数原型如下：



```
string&insert (size_t pos, const string&str);    // 在位置 pos 处插入字符串 str

string&insert (size_t pos, const string&str, size_t subpos, size_t sublen); // 在位置 pos 处插入字符串 str 的从位置 subpos 处开始的 sublen 个字符

string&insert (size_t pos, const char * s);      // 在位置 pos 处插入字符串 s

string&insert (size_t pos, const char * s, size_t n); // 在位置 pos 处插入字符串 s 的前 n 个字符

string&insert (size_t pos, size_t n, char c);    // 在位置 pos 处插入 n 个字符 c

iterator insert (const_iterator p, size_t n, char c); // 在 p 处插入 n 个字符 c，并返回插入后迭代器的位置

iterator insert (const_iterator p, char c);      // 在 p 处插入字符 c，并返回插入后迭代器的位置
```



举个例子：



```
#include <string>
#include <iostream>

using namespace std;

int main()
```

```

{
    string str("abcdefgh");
    str.insert(1, "INSERT");           // 在位置 1 处插入字符串 "INSERT"
    cout << str << endl;

    str.insert(10, 5, 'A');           // 在位置 10 处插入 5 个字符 'A'
    cout << str << endl;
    return 0;
}

```



输出结果如下：



Microsoft Visual Studio 调试控制台

```

aINSERTbcdefgh
aINSERTbcdAAAAAefgh

```

## 9.2 使用 `string.erase()` 进行元素删除操作

函数原型如下：

```

string& erase (size_t pos = 0, size_t len = npos);           // 删除从 pos 处开始的 n 个字符

iterator erase (const_iterator p);                           // 删除 p 处的一个字符，并返回删除后迭代器的位置

iterator erase (const_iterator first, const_iterator last); // 删除从 first 到 last 之间的字符，并返回删除后迭代器的位置

```

举个例子：



```

#include <string>
#include <iostream>

```

```
using namespace std;

int main()
{
    string str("Hello,World!");
    str.erase(5,6);           // 删除从索引位置 5 开始的 6 个字符
    cout << "str 为: " << str << endl;

    return 0;
}
```



关于 erase() 函数的用法可以参考 <https://www.cnblogs.com/liyazhou/archive/2010/02/07/1665421.html>

## 十、string 对象的一些其他操作

### 10.1 使用 getline() 函数来获取 string 输入

```
string str;

getline(cin,str);    // 从输入流中读取一行数据到 str
```

### 10.2 使用 empty() 函数判断字符串是否为空

```
string str;

if(str.empty()){
    cout << "字符串为空" << endl;
}
```

string.empty() 函数，若字符串为空，则返回真，否则返回假。

### 10.3 使用 swap 函数交换两个字符串



```
#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str1 = "hello,world!";
    string str2 = "HELLO,WORLD!";

    str1.swap(str2);

    cout << str1 << endl;
    cout << str2 << endl;

    return 0;
}
```



参考资料：